

# **Hotel Reservation Database System**

**B103 Databases and Big Data**

**George Topuria**

**GH1045086**

**Github:**

**<https://github.com/georgetopuria/Hotel-Reservation-Database-System>**

**Video Demonstration:**

**<https://youtu.be/TZCFDQb-QVE>**

# **1 introduction**

## **1.1 project overview**

The main goal of this project is to design and demonstrate a hotel reservation database system using SQL. Hotels have to manage huge amounts of information regarding Rooms guest payments, reservations, services and more, so a well designed database is necessary to keep the information stored efficiently and consistently without extra steps and repetitions.

The hotel reservation database system was made to give a structured and effective way of managing basic hotel operations. This system allows the staff of the hotel to record information about guests, room types, payments and services during their stay.

## **1.2 project objectives**

Our main objectives for this project are to:

Be able to hold hotel reservation data efficiently, and the ease of use to create a smooth list process.

Reduce data duplication, meaning unwanted repetitions of the same data.

Yo manage Rooms bookings, manage guests, and services.

Use SQL to create update insert delete and querying data.

Show joints, aggregations, triggers, stored procedures, constraints, and transactions.

## **1.3 chosen domain justification**

The reason the hotel reservation system was chosen is because it is a realistic business environment with many entities and many relationships. This domain consist of guest, rooms,

bookings, payments and services making it perfect for demonstrating the abilities and functionality of SQL.

## **2 system design**

### **2.1 main entities/tables**

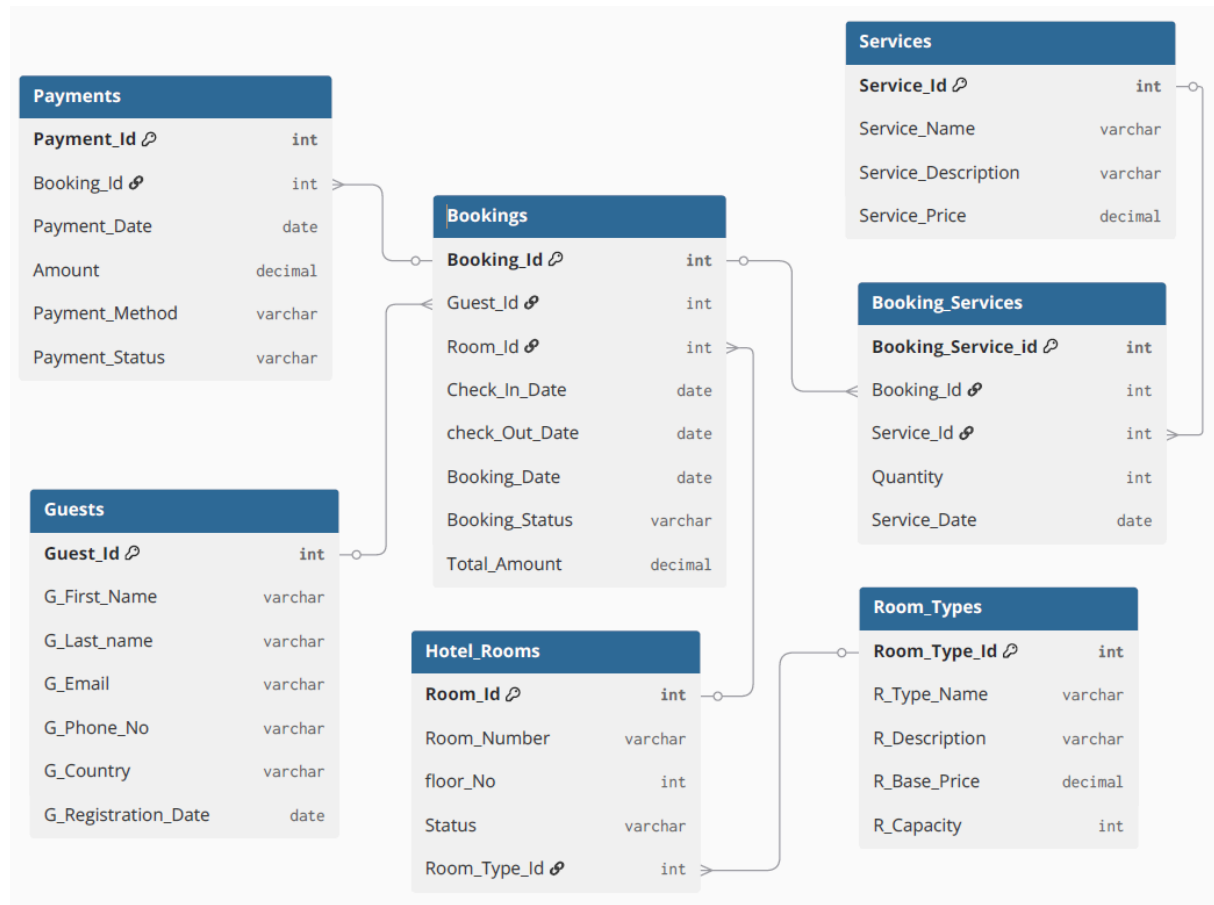
For this project, I made seven tables which will perfectly demonstrate the capabilities of our SQL and eliminate unnecessary repetition.

Guest table, this table stores guests, personal details, such as first and last names, phone numbers, email addresses, and registration dates. Room types table holds data about the room types, such as the name of the room, the description of the room, the base price, and the capacity. The room table holds information about specific rooms, such as room number, floor number, room status, and the room type. The bookings table stores all the information for reservations, such as the customer IDs room, check-in and check-out dates, booking dates, booking statuses, and the total price. The payments table holds information about the payments, such as the booking ID, payment date, amount, payment method, and status. The services table stores hotel services information such as the service name, the description of the service, and the price. And the booking services table holds information about the service ID, the booking ID, the quantity of service used, and the date the service was used.

### **2.2 ER diagram**

This ER diagram shows the main tables, entities attributes, primary and foreign keys used in the hotel reservation database.

This ER diagram was made with dbdiagram.io



## 2.3 relationship relationships

Here are the relationship relationships shown on the ER diagram:

One guest can make many bookings.

One room type can describe many different hotel rooms.

One hotel room can appear in multiple bookings overtime.

One booking and have multiple payments.

One booking can have multiple services.

One service can be used in multiple bookings.

And the booking services table is there to solve the many to many relationship between bookings and services.

### **3 normalization and design justification**

We have to abide by normalization because it is the specific guideline that helps us avoid repetition and fix errors/anomalies that we would have otherwise

#### **3.1 first normal form**

The database definitely satisfies the first normal form because each cell contains one value meaning no column stores. Multiple types of information and all records are identified uniquely through their primary keys.

#### **3.2 second normal form**

The database definitely satisfies the second normal form because the non-attributes are completely dependent on their respective primary keys. Guest information room, information booking information and payment information have their own dedicated tables.

#### **3.3 third normal form**

The database definitely satisfies the third normal form because non-attributes are not dependent on other non-attributes for example, room, descriptions or room prices are stored in the room types table instead of being duplicated in every room. This improves consistency and help us maintain efficiency without having repetitive information.

### **4 implementation**

## 4.1 database creation

The database was created using the SQL create payable statement. Each table has his own appropriate primary key and has an established relationship with the foreign keys.

## 4.2 table creation

The main SQL features that were used to create tables was as follows;

**Create table:** for creating the table

**Primary key:** to signify that it is a primary key

**Foreign key:** to reference a primary key in another table

**Not null:** to make sure that the value is never null

**Unique:** to make sure it doesn't match with other units in the table

**Check:** to make sure that I will cross reference with a given constraint

**Default:** will signify to show a default message, if not told otherwise

**Auto increment:** to automatically assign a value that increases by one

**Varchar:** to be able to create a string limited to a certain number of characters

These are samples from the code being executed. Only the Guests and Bookings tables are displayed because they showcase; foreign keys, primary keys and constraints.

| Guest_Id | First_Name | Last_name | Email          | Phone_No   | Country | Registration_Date |
|----------|------------|-----------|----------------|------------|---------|-------------------|
| 1        | Nika       | Japaridze | Nika@japar.com | 1236769420 | Georgia | 2026-06-24        |
| 2        | Gio        | Topuria   | Gio@topka.com  | 6767676767 | Georgia | 2026-06-24        |
| 3        | Zuka       | Mujiri    | Zuka@mujir.com | 420420420  | Georgia | 2026-06-24        |
| 4        | Chelo      | Iradze    | Chelo@irad.com | 60606060   | Georgia | 2026-06-24        |

| Booking_Id | Guest_Id | Room_Id | Check_In_Date | Check_Out_Date | Booking_Date | Booking_Status | Total_Amount |
|------------|----------|---------|---------------|----------------|--------------|----------------|--------------|
| 1          | 1        | 1       | 2026-07-02    | 2026-07-04     | 2026-04-01   | Pending        | 100.00       |
| 2          | 2        | 10      | 2026-06-02    | 2026-06-22     | 2026-02-10   | Confirmed      | 2360.00      |
| 3          | 3        | 4       | 2026-04-02    | 2026-04-08     | 2026-01-20   | Completed      | 185.00       |
| 4          | 4        | 5       | 2026-06-09    | 2026-06-15     | 2026-02-01   | Cancelled      | 150.00       |

### 4.3 constraints

Some of the constraints used in our database are unique such as guest emails have to be unique, not null for example a room price must be greater than zero, check for for example, a booking check out date must be after the check-in date, varchar for example a booking status is limited to accept values that have been predetermined.

### 5 sample data

I inserted sample data into the guest room types, hotel rooms, bookings, payments services and booking services tables. The data was inserted in a specific order because of the foreign keys that require records to exist in the first place. For example, guests and room types were inserted because of bookings since bookings depended on already existing guests and Rooms.

The inserted data helps us demonstrate a realistic hotel operation with multiple room, types, guests, services and handling different bookings and payments

### 6 queries and results

After creating the database and inserting sample data, multiple SQ queries were developed to show data retrieval, filtering, sorting aggregation, and integration through joints

#### 6.1 basic retrieval queries

For example, show all guests or show all hotel rooms or show all bookings.

This table demonstrates a retrieval queries showing all guests by filtering countries. In this case it was asked to show all from Georgia.

```
SELECT *  
  
from Guests  
  
WHERE Country = 'Georgia';
```

| Guest_Id | First_Name | Last_name | Email           | Phone_No   | Country | Registration_Date |
|----------|------------|-----------|-----------------|------------|---------|-------------------|
| 1        | Nika       | Japaridze | Nika@japar.com  | 1236769420 | Georgia | 2026-06-24        |
| 2        | Gio        | Topuria   | Gio@topka.com   | 6767676767 | Georgia | 2026-06-24        |
| 3        | Zuka       | Mujiri    | Zuka@mujiir.com | 420420420  | Georgia | 2026-06-24        |
| 4        | Chelo      | Iradze    | Chelo@irad.com  | 60606060   | Georgia | 2026-06-24        |

## 6.2 fitted queries

For example, show available rooms show guests from Georgia show confirmed bookings,

The table here demonstrates the fitted querie retrieving all rooms where the status is “Available”.

```
SELECT *  
  
from hotel_rooms  
  
WHERE Status = 'Available';
```

| Room_Id | Room_No | floor | Status    | Room_Type_Id |
|---------|---------|-------|-----------|--------------|
| 3       | 112     | 1     | Available | 3            |
| 6       | 211     | 2     | Available | 6            |
| 7       | 212     | 2     | Available | 7            |
| 8       | 310     | 3     | Available | 8            |
| 9       | 311     | 3     | Available | 9            |

## 6.3 short inquiries



For example, room types by price that is descending or show gas alphabetically by their last names

This table demonstrates the shoty queries listing rooms by price descending.

```
select *
```

```
from Room_Types
```

```
Order BY room_Price DESC;
```

| Room_Type_Id | Room_Type_Name | Description                | Room_Price | Capacity |
|--------------|----------------|----------------------------|------------|----------|
| 10           | presidencial   | biggest suite in the hotel | 300.00     | 7        |
| 8            | family room    | a room with three beds     | 180.00     | 4        |
| 9            | family room    | a room with three beds     | 180.00     | 4        |
| 5            | double room    | a room with two beds       | 100.00     | 2        |
| 6            | double room    | a room with two beds       | 100.00     | 2        |
| 7            | double room    | a room with two beds       | 100.00     | 2        |
| 1            | single room    | a room with one bed        | 70.00      | 1        |
| 2            | single room    | a room with one bed        | 70.00      | 1        |
| 3            | single room    | a room with one bed        | 70.00      | 1        |
| 4            | single room    | a room with one bed        | 70.00      | 1        |

## 6.4 aggregation queries

For example count the total revenue or find the most expensive room type or count total guests or calculate the average room prices.

This table demonstrates the aggregation queries listing the total revenue.

```
select SUM(Amount) AS Revenue
```

```
FROM Payments;
```

| Revenue |
|---------|
| 2835.00 |

## 6.5 join queries

For example show guest names with their bookings or show guest payment information or show services used in each booking or show bookings with room type and room.

This table demonstrates the join queries creating a table joining the guests and bookings tables; and another table by joining bookings, room and room types.

```
select
Guests.First_Name,
Guests.Last_Name,
bookings.booking_id,
bookings.check_in_date,
bookings.check_out_Date,
bookings.booking_status
From guests
join Bookings
on Guests.Guest_Id = Bookings.Guest_Id;
```

| First_Name | Last_Name | booking_id | check_in_date | check_out_Date | booking_status |
|------------|-----------|------------|---------------|----------------|----------------|
| Nika       | Japaridze | 1          | 2026-07-02    | 2026-07-04     | Pending        |
| Gio        | Topuria   | 2          | 2026-06-02    | 2026-06-22     | Confirmed      |
| Zuka       | Mujiri    | 3          | 2026-04-02    | 2026-04-08     | Completed      |
| Chelo      | Iradze    | 4          | 2026-06-09    | 2026-06-15     | Cancelled      |

| Booking_Id | Room_No | Room_Type_Name | Check_In_Date | Check_Out_Date |
|------------|---------|----------------|---------------|----------------|
| 1          | 110     | single room    | 2026-07-02    | 2026-07-04     |
| 2          | 410     | presidencial   | 2026-06-02    | 2026-06-22     |
| 3          | 113     | single room    | 2026-04-02    | 2026-04-08     |
| 4          | 210     | double room    | 2026-06-09    | 2026-06-15     |

## 6.6 Grouping and having queries

For example, count bookings by status or revenue by payment method or counting rooms by type or payment methods with revenue greater than 100.’

This table demonstrates the grouping and having queries making a table with booking status and number of bookings with the same status.

Select

booking\_status,

count(\*) as number\_of\_bookings

from bookings

group by booking\_status;

| booking_status | number_of_bookings |
|----------------|--------------------|
| Pending        | 1                  |
| Confirmed      | 1                  |
| Completed      | 1                  |
| Cancelled      | 1                  |

The queries that have been implemented shows that the database was successful at:

- Data retrieval
- Data filtering

- Data sorting
- Group analysis
- Multi table joints
- Data aggregation

The results have shown that the database and the relationships are functioning as intended

## 7 advanced SQL features

To show furthermore, the capabilities of the database, some advanced SQL concepts were demonstrated.

### 7.1 stored procedure

The stored procedure is a procedure where `getguestbookings` retrieves all bookings for a selected guest ID. Stored procedures are useful because they are reusable highly efficient and very modular meaning it can convert to any situation.

```
238 Delimiter ;
239
240 insert into payments
241 (booking_id, payment_Date, amount, method, Payment_status)
242 VALUES
243 (1, '2026-04-02', 50.00, 'Card', TRUE);
244
245 Select * FROM Bookings;
246
247 Call GetGuestBookings(2);
```

### 7.2 trigger

A trigger is an automatic SQL action that is executed after the insert update or delete. The trigger I have installed makes it so a booking is updated to completed when a payment goes through

successfully, thus it will not have to be changed manually, making it more efficient than having to type it out yourself.

```
225 Delimiter //
226
227 CREATE trigger updatebookingafterpayment
228 after Insert on Payments
229 for each row
230 Begin
231     IF NEW.Payment_Status = TRUE THEN
232         Update bookings
233         Set booking_Status = 'Completed'
234         Where booking_id = NEW.booking_Id;
235     end if;
236 end //
237
238 Delimiter ;
```

| Booking_Id | Guest_Id | Room_Id | Check_In_Date | Check_Out_Date | Booking_Date | Booking_Status | Total_Amount |
|------------|----------|---------|---------------|----------------|--------------|----------------|--------------|
| 1          | 1        | 1       | 2026-07-02    | 2026-07-04     | 2026-06-01   | Completed      | 100.00       |
| 2          | 2        | 15      | 2026-08-02    | 2026-08-22     | 2026-02-10   | Confirmed      | 2360.00      |
| 3          | 3        | 4       | 2026-04-02    | 2026-04-18     | 2026-01-20   | Completed      | 185.00       |
| 4          | 4        | 5       | 2026-08-09    | 2026-08-15     | 2026-02-01   | Cancelled      | 190.00       |

### 7.3 transaction

A transaction is an ACID property, ACID stand, stands for automatically consistently, isolation and durability. The transaction inserts a payment and then updates the related booking status at the same time, if the operation happens to fail, it has the option to be reset to maintain the database consistency.

```
249 START transaction;
250
251 insert into payments
252 (Booking_Id, payment_date, amount, Method, payment_Status)
253 VALUES
254 (2, '2026-06-01', 500.00, 'Card', TRUE);
255
256 update bookings
257 set booking_Status = 'completed'
258 where Booking_id = 2;
259
260 Commit;
```

## **8 challenges and solutions**

### **8.1 foreign key order**

One problem, I faced was that bookings could not be inserted before guests and Rooms existed and to figure this out I had to go through multiple errors and inconsistent tables.

The solution for this was to insert parent tables 1st which in this case was guest and room types before inserting my booking table.

### **8.2 foreign key placement**

One of the biggest challenges, I faced was implementing foreign keys in tables, I could not figure out how to implement a foreign key within a different table and how to tell my database to refer to a primary key in a different table.

The solution was with SQL, I had to use refer to make my SQL understand that I was referring to a different primary key.

## **9 conclusion and future work**

### **9.1 conclusion**

The project successfully was able to design and implement a hotel reservation database using the SQL format. It confines within rules such as normalization and demonstrates constraints relationships, queries data, inserts, joints, triggers, stored procedures, and transactions.

The database was capable to support

- Payment processing

- Booking processing
- Room processing
- Guest management
- Services

And the project was confiding in concepts such as:

- Relational database design
- Normalization
- Key constraints
- Foreign and primary keys
- SQL queries both basic and advanced
- Joins
- Triggers
- Start procedures
- Transactions

With all these demonstrations, the project shows that it could be heavily reliable in a real world scenario.

## **9.2 future work**

For this project to be used in a real life scenario, it would benefit with some enhancements.

Possible improvements could include:

- A staff table
- Room availability calendar
- A loyalty system for the customers
- Refund/cancellation logic
- Indexes for more efficient searching
- A more realistic payment status

These improvements would make the database even more efficient and ready for the real system.

## 10 reference

### Works Cited

Connolly, Thomas, and Carolyn Begg. *GloBal EDITION Database Systems a Practical Approach to Design, Implementation, and Management SIXTH EDITION*.

“Dbdiagram.io - Database Relationship Diagrams Design Tool.” *Dbdiagram.io*, 2026, dbdiagram.io/d/DBMS-6a26d04325fc5bf036ba3be5. Accessed 24 June 2026.

“FREE AI-Enhanced Online MySQL Compiler - for Learning & Practice.” *Sqlfiddle.com*, 2024, sqlfiddle.com/mysql/online-compiler?id=01e99d0c-2cdb-495d-b29b-ceedd95874f5. Accessed 24 June 2026.

Microsoft. “Visual Studio Code.” *Visualstudio.com*, Microsoft, 2025, code.visualstudio.com/.

Oracle. “MySQL.” *Mysql.com*, Oracle, 2024, www.mysql.com/.

*Physical Database Design*.